DR. RVR & NRI INSTITUTE OF TECHNOLOGY

PYTHON PROGRAMMING

ACTIVITY-BASED LEARNING (ABL) ASSIGNMENT

10 Selected Case Studies with Algorithms, Flowcharts, Pseudocode, Python Programs and Execution Evidence

Student Name	_____
Roll Number	_____
Branch / Section	_____
Course	Python Programming
Academic Activity	Dussehra Holiday ABL Assignment

Submitted to: Python Programming Faculty Member / Lab Mentor

Submission date: _____

Assignment Overview

This report documents ten Python case studies selected for the Python Programming Activity-Based Learning assignment. Each case study includes the problem statement, objectives, assumptions, algorithm, a text flowchart, pseudocode, Python source code, test cases, execution evidence, and a conclusion.

The execution screenshots are included as supplied. Where a screenshot demonstrates only one branch (for example, an invalid menu choice), the test-case table also lists additional expected test scenarios; those additional scenarios are not represented as executed screenshots.

Contents

01	Electricity Billing System
02	Product Billing with Discount
03	Attendance Eligibility Calculator
04	Bank Account Menu: Deposit, Withdraw and Balance
05	Exam Marks, Performance and Grade
06	Shopping Budget Checker
07	Number Pattern Using Nested Loops
08	Password Validation
09	Student Marks and Top Performer
10	Canteen Ordering and Stock Management

Note: The source programs are transcribed from the supplied editor screenshots. Case 4 and Case 10 include the complete menu branches reconstructed from the visible code and demonstrated behavior so the report contains runnable full programs.

CASE STUDY 01: Electricity Billing System

1. Problem Statement

Develop a program that calculates an electricity bill from the number of units consumed. Apply slab rates: Rs. 2 per unit for the first 100 units, Rs. 3 per unit for units 101–200, and Rs. 5 per unit above 200. Reject negative unit values.

2. Objectives

• Read electricity consumption from the user.
• Apply progressive slab-wise billing.
• Display the calculated bill or an invalid-input message.

3. Assumptions

• The number of units is a whole number.
• Rates are applied progressively to each slab.
• No fixed charge, tax, or surcharge is included.

4. Algorithm

1. Start and read units consumed.
2. If units are negative, display an invalid-units message.
3. If units are at most 100, calculate units × 2.
4. If units are at most 200, calculate 100 × 2 + (units – 100) × 3.
5. Otherwise calculate 100 × 2 + 100 × 3 + (units – 200) × 5.
6. Display the bill and stop.

5. Flowchart

```
START → Read units → units < 0? → Yes: Invalid units → END
                          No → units ≤ 100? → Yes: bill = units × 2
                                                  No → units ≤ 200? → Yes: bill = 200 + (units−100) × 3
                                                                      No: bill = 500 + (units−200) × 5
→ Display bill → END
```

6. Pseudocode

```
BEGIN
  INPUT units
  IF units < 0 THEN
    PRINT invalid units
  ELSE IF units <= 100 THEN
    bill = units * 2
  ELSE IF units <= 200 THEN
    bill = 100 * 2 + (units - 100) * 3
  ELSE
    bill = 100 * 2 + 100 * 3 + (units - 200) * 5
  END IF
  IF units >= 0 THEN PRINT bill
END
```

7. Python Source Code

```
print("Electricity Billing System")
units = int(input("Enter electricity units: "))
if units < 0:
    print("Invalid units")
elif units <= 100:
    bill = units * 2
elif units <= 200:
    bill = 100 * 2 + (units - 100) * 3
else:
    bill = 100 * 2 + 100 * 3 + (units - 200) * 5

if units >= 0:
    print("Electricity Bill:", bill)
```

8. Test Cases

Input / Scenario	Expected Result	Evidence in supplied run
200	Rs. 500	Rs. 500 (shown)
50	Rs. 100	Not shown
-5	Invalid units	Not shown
250	Rs. 750	Not shown

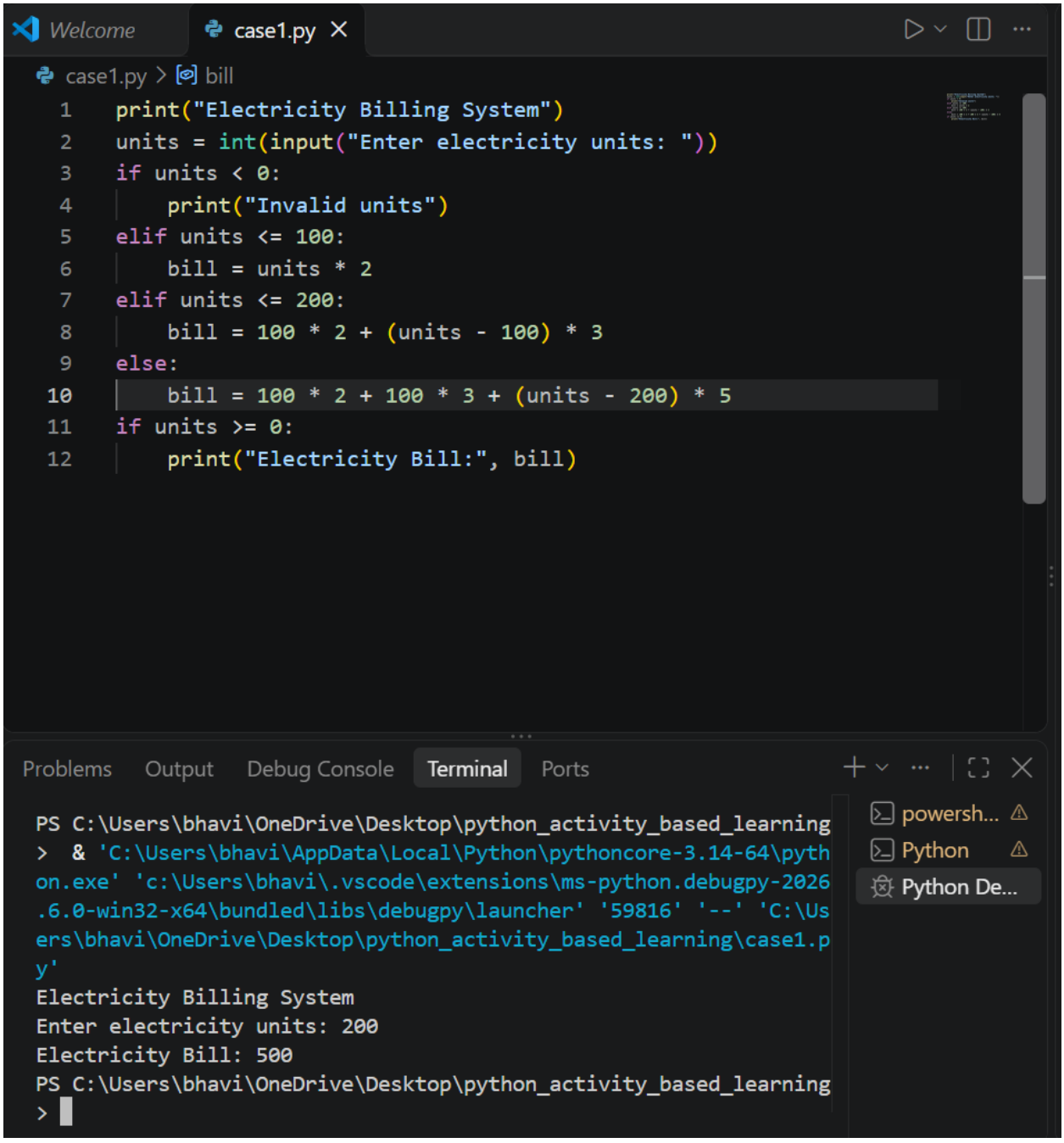
10. Conclusion

The program uses conditional statements to calculate a progressive electricity bill and prevents negative consumption values from being billed.

CASE STUDY 01: EXECUTION OUTPUT

9. Output Screenshot

The following image is the execution evidence supplied for this case study.



CASE STUDY 02: Product Billing with Discount

1. Problem Statement

Create a billing program that accepts a product's unit price and quantity, calculates the total amount, applies a 10% discount when the total is Rs. 1,000 or more, and displays the final bill.

2. Objectives

• Accept price and quantity.
• Calculate the gross total.
• Apply the discount threshold and print the final amount.

3. Assumptions

• Price is a non-negative decimal value.
• Quantity is a positive whole number.
• The discount is 10% of the total when total ≥ Rs. 1,000.

4. Algorithm

1. Read product price and quantity.
2. Calculate total = price × quantity.
3. If total is at least 1,000, set discount to 10% of total; otherwise set discount to zero.
4. Calculate final bill = total – discount.
5. Display total, discount, and final bill.

5. Flowchart

```
START → Read price, quantity → total = price × quantity
→ total ≥ 1000? → Yes: discount = total × 0.10
                  No: discount = 0
→ final bill = total − discount → Display amounts → END
```

6. Pseudocode

```
BEGIN
  INPUT price, quantity
  total = price * quantity
  IF total >= 1000 THEN
    discount = total * 0.10
  ELSE
    discount = 0
  END IF
  final_bill = total - discount
  PRINT total, discount, final_bill
END
```

7. Python Source Code

```
price = float(input("Enter product price: "))
quantity = int(input("Enter quantity: "))
total = price * quantity

if total >= 1000:
    discount = total * 0.10
else:
    discount = 0

final_bill = total - discount
print("Total Amount:", total)
print("Discount:", discount)
print("Final Bill:", final_bill)
```

8. Test Cases

Input / Scenario	Expected Result	Evidence in supplied run
500 × 3	Total 1500; discount 150	Final bill 1350 (shown)
200 × 2	Total 400; discount 0	Not shown
1000 × 1	Total 1000; discount 100	Not shown

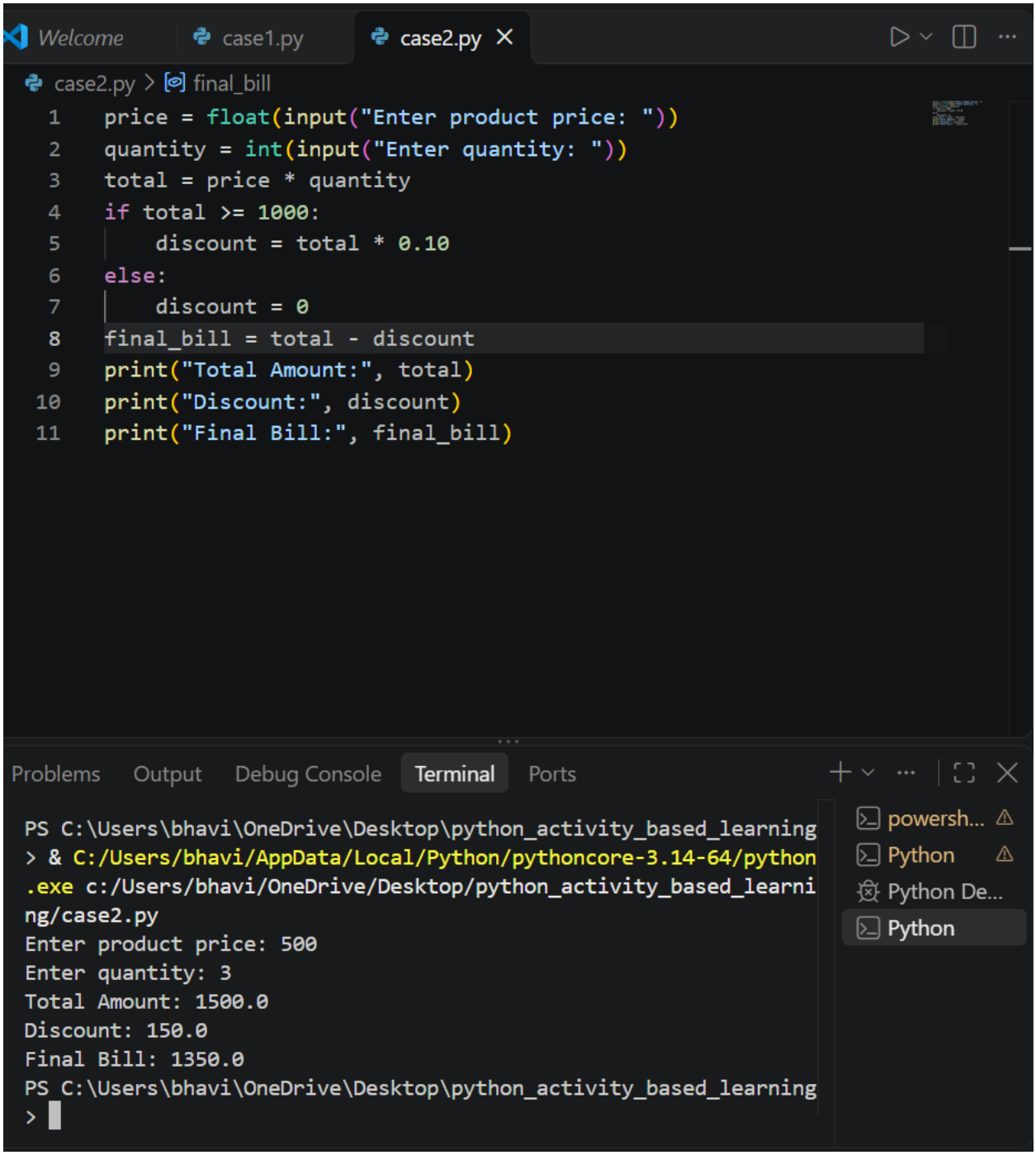
10. Conclusion

The program demonstrates arithmetic operations and if-else decision making to calculate a discount and final payable amount.

CASE STUDY 02: EXECUTION OUTPUT

9. Output Screenshot

The following image is the execution evidence supplied for this case study.



CASE STUDY 03: Attendance Eligibility Calculator

1. Problem Statement

Calculate a student's attendance percentage and determine examination eligibility. If attendance is below 75%, calculate the minimum number of additional classes the student must attend, assuming every future class attended increases both attended and total classes by one.

2. Objectives

• Read attended and total class counts.
• Validate attendance details.
• Calculate percentage and eligibility.
• Use a while loop to find additional classes required.

3. Assumptions

• Class counts are whole numbers.
• Total classes must be positive.
• Attended classes cannot be negative or exceed total classes.
• Every additional class is attended.

4. Algorithm

1. Read attended classes and total classes.
2. Validate total > 0 and 0 ≤ attended ≤ total.
3. Calculate percentage = attended / total × 100.
4. If percentage ≥ 75, display eligibility.
5. Otherwise start extra at zero and increment it until (attended + extra)/(total + extra) × 100 reaches 75.
6. Display the number of extra classes.

5. Flowchart

```
START → Read attended, total → Validate values
→ Invalid? → Yes: Print invalid details → END
           No: Calculate percentage → percentage ≥ 75?
                 Yes: Eligible → END
                 No: extra = 0 → WHILE projected percentage < 75: extra += 1
                     → Print extra classes → END
```

6. Pseudocode

```
BEGIN
  INPUT attended, total
  IF total <= 0 OR attended < 0 OR attended > total THEN
    PRINT invalid attendance details
  ELSE
    percentage = attended / total * 100
    PRINT percentage
    IF percentage >= 75 THEN
      PRINT eligible
    ELSE
      extra = 0
      WHILE (attended + extra) / (total + extra) * 100 < 75
        extra = extra + 1
      END WHILE
      PRINT extra
    END IF
  END IF
END
```

7. Python Source Code

```
attended = int(input("Enter classes attended: "))
total = int(input("Enter total classes: "))

if total <= 0 or attended < 0 or attended > total:
    print("Invalid attendance details")
else:
    percentage = attended / total * 100
    print("Attendance Percentage:", percentage)

    if percentage >= 75:
        print("Eligible for examination")
    else:
        extra = 0
        while (attended + extra) / (total + extra) * 100 < 75:
            extra += 1
        print("Extra classes required:", extra)
```

8. Test Cases

Input / Scenario	Expected Result	Evidence in supplied run
30 attended / 50 total	60%; 30 extra classes	30 extra classes (shown)
40 / 50	80%; eligible	Not shown
55 / 50	Invalid details	Not shown

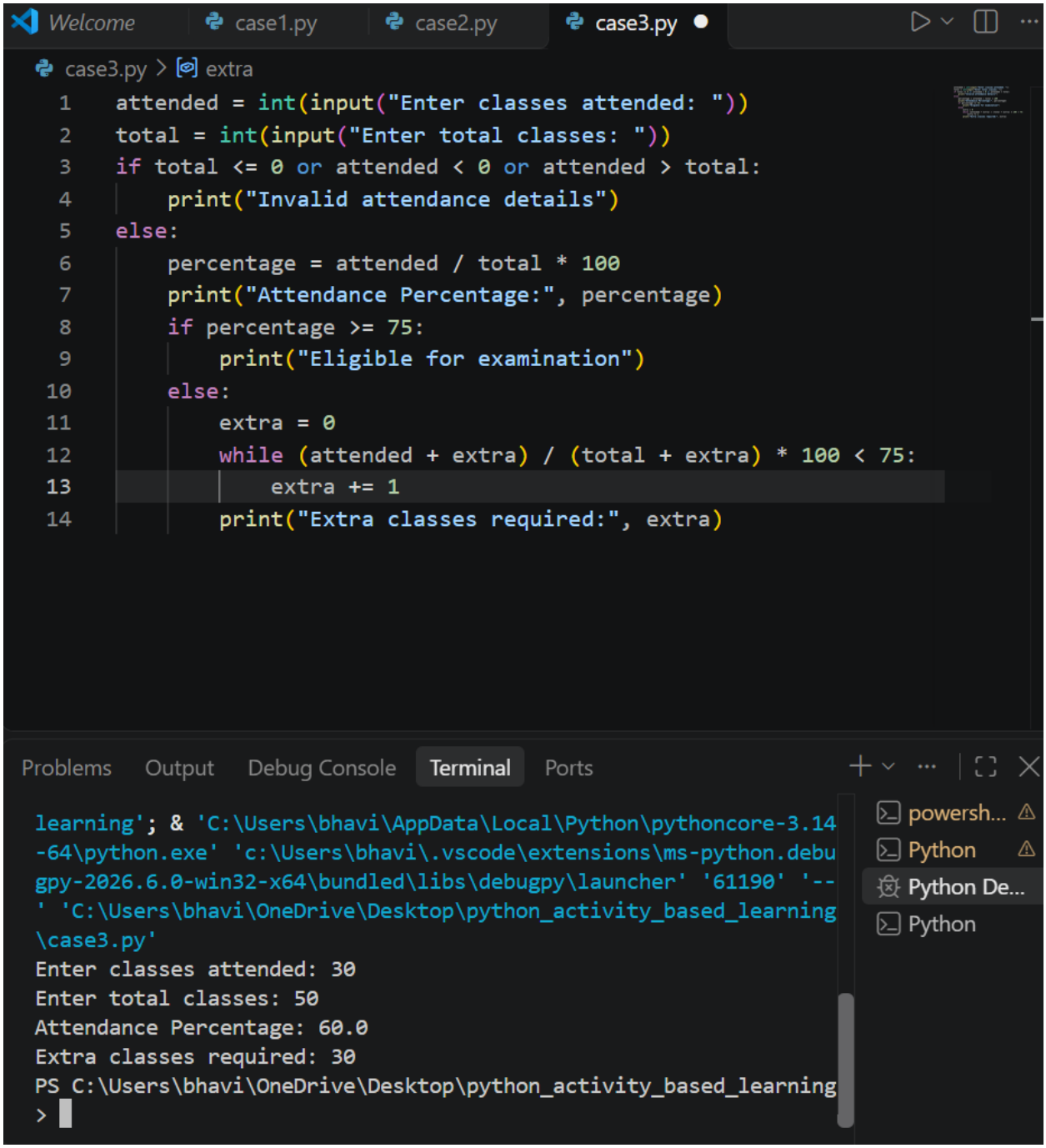
10. Conclusion

The program combines input validation, percentage calculation, nested conditions, and a while loop to determine examination eligibility.

CASE STUDY 03: EXECUTION OUTPUT

9. Output Screenshot

The following image is the execution evidence supplied for this case study.



CASE STUDY 04: Bank Account Menu: Deposit, Withdraw and Balance

1. Problem Statement

Develop a simple banking menu that allows a user to deposit money, withdraw money, or check the account balance. Validate transaction amounts and prevent withdrawals greater than the available balance.

2. Objectives

• Accept an opening balance.
• Provide deposit, withdrawal, and balance options.
• Validate transaction amounts and available funds.
• Display the final account balance.

3. Assumptions

• The opening balance and transaction amounts are numeric.
• Only one menu operation is performed per execution.
• A transaction amount must be greater than zero.
• No interest, fees, or overdraft facility is included.

4. Algorithm

1. Read the initial balance and display the menu.
2. Read the user's choice.
3. For deposit, accept a positive amount and add it to the balance.
4. For withdrawal, accept a positive amount and ensure it does not exceed the balance.
5. For balance enquiry, display the balance.
6. For any other choice, display Invalid Choice.
7. Display the final balance.

5. Flowchart

```
START → Read opening balance → Display menu → Read choice
→ Choice = 1? Deposit and validate amount
→ Else choice = 2? Withdraw; validate amount and funds
→ Else choice = 3? Display balance
→ Else: Invalid choice → Display final balance → END
```

6. Pseudocode

```
BEGIN
  INPUT balance
  DISPLAY menu: 1 Deposit, 2 Withdraw, 3 Check Balance
  INPUT choice
  IF choice = 1 THEN
    INPUT amount
    IF amount > 0 THEN balance = balance + amount
    ELSE PRINT invalid amount
  ELSE IF choice = 2 THEN
    INPUT amount
    IF amount <= 0 THEN PRINT invalid amount
    ELSE IF amount <= balance THEN balance = balance - amount
    ELSE PRINT insufficient balance
  ELSE IF choice = 3 THEN
    PRINT balance
  ELSE
    PRINT invalid choice
  END IF
  PRINT final balance
END
```

7. Python Source Code

```
balance = float(input("Enter initial balance: "))
print("1. Deposit")
print("2. Withdraw")
print("3. Check Balance")
choice = int(input("Enter your choice: "))

if choice == 1:
    amount = float(input("Enter deposit amount: "))
    if amount > 0:
        balance = balance + amount
        print("Deposit Successful")
    else:
        print("Invalid amount")
elif choice == 2:
    amount = float(input("Enter withdrawal amount: "))
    if amount <= 0:
        print("Invalid amount")
    elif amount <= balance:
        balance = balance - amount
        print("Withdrawal Successful")
    else:
        print("Insufficient balance")
elif choice == 3:
    print("Available Balance:", balance)
else:
    print("Invalid Choice")

print("Final Balance:", balance)
```

8. Test Cases

Input / Scenario	Expected Result	Evidence in supplied run
Opening 5000; choice 2000	Invalid choice; final balance 5000	Invalid Choice; final balance 5000 (shown)
5000; deposit 500	Deposit successful; balance 5500	Not shown
5000; withdraw 6000	Insufficient balance; balance 5000	Not shown
5000; choice 3	Balance 5000	Not shown

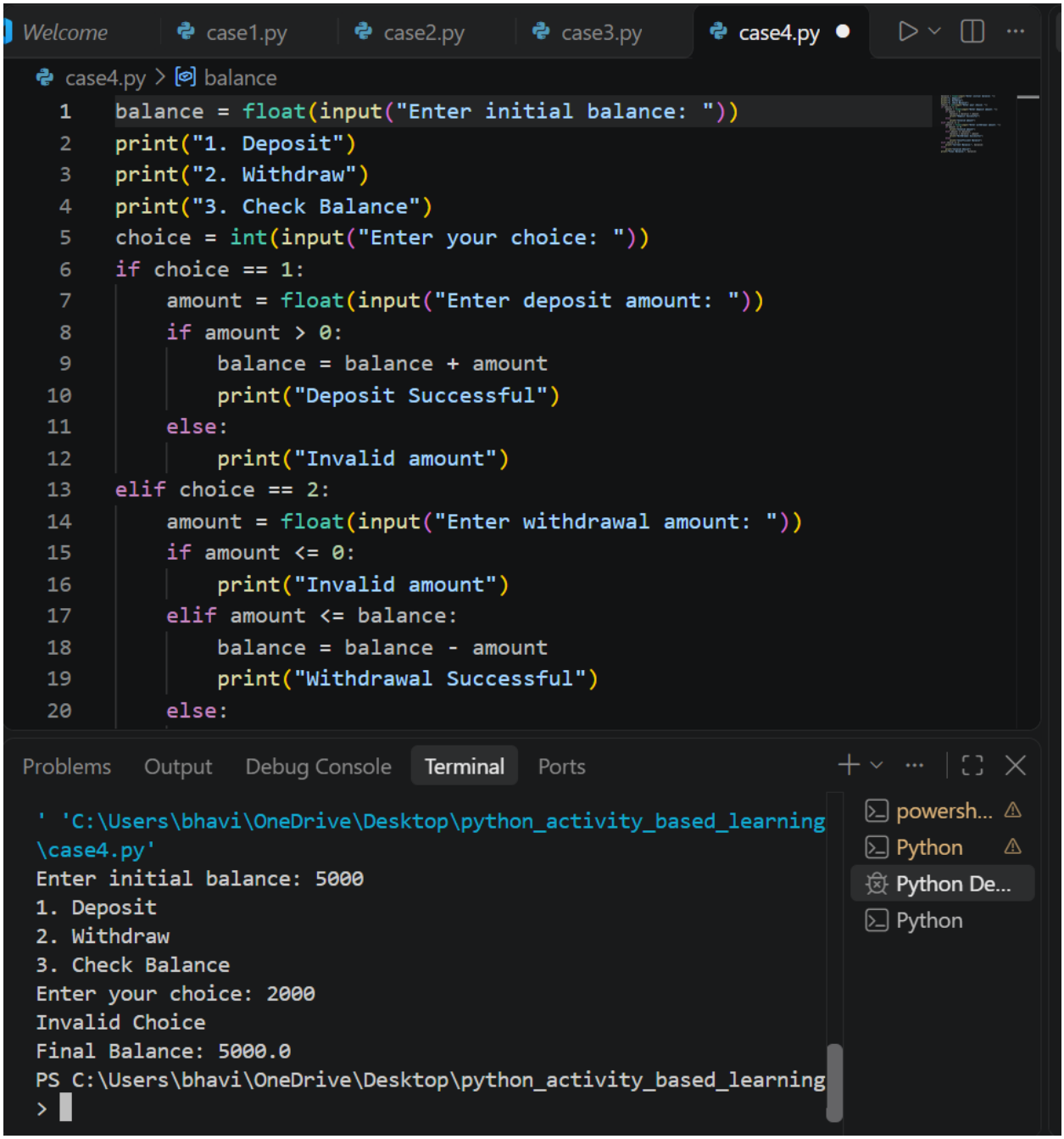
10. Conclusion

The menu-driven program demonstrates conditional branching and basic account operations while checking transaction validity. The supplied run tests the invalid-choice branch.

CASE STUDY 04: EXECUTION OUTPUT

9. Output Screenshot

The following image is the execution evidence supplied for this case study.



CASE STUDY 05: Exam Marks, Performance and Grade

1. Problem Statement

Read marks in three examinations, calculate the average, compare the third examination mark with the first to identify performance change, and assign a grade based on the average.

2. Objectives

• Read three examination marks.
• Calculate and display the average.
• Compare first and third examination marks.
• Assign a grade using average thresholds.

3. Assumptions

• Each mark is numeric and is expected to be between 0 and 100.
• All three examinations have equal weight.
• Grade thresholds: 75 distinction, 60 first class, 40 pass, below 40 fail.

4. Algorithm

1. Read marks for examinations one, two, and three.
2. Calculate average = (mark1 + mark2 + mark3) / 3.
3. Compare mark3 with mark1 and display improved, declined, or remained same.
4. If average ≥ 75, display distinction.
5. Else if average ≥ 60, display first class; else if average ≥ 40, display pass; otherwise display fail.

5. Flowchart

```
START → Read 3 marks → Calculate average → Compare mark3 with mark1
→ Print performance trend → average ≥ 75? Distinction
  Else average ≥ 60? First Class
  Else average ≥ 40? Pass
  Else Fail → END
```

6. Pseudocode

```
BEGIN
  INPUT mark1, mark2, mark3
  average = (mark1 + mark2 + mark3) / 3
  PRINT average
  IF mark3 > mark1 THEN PRINT improved
  ELSE IF mark3 < mark1 THEN PRINT declined
  ELSE PRINT remained same
  IF average >= 75 THEN PRINT distinction
  ELSE IF average >= 60 THEN PRINT first class
  ELSE IF average >= 40 THEN PRINT pass
  ELSE PRINT fail
END
```

7. Python Source Code

```
mark1 = float(input("Enter first exam marks: "))
mark2 = float(input("Enter second exam marks: "))
mark3 = float(input("Enter third exam marks: "))

average = (mark1 + mark2 + mark3) / 3
print("Average Marks:", average)

if mark3 > mark1:
    print("Performance Improved")
elif mark3 < mark1:
    print("Performance Declined")
else:
    print("Performance Remained Same")

if average >= 75:
    print("Grade: Distinction")
elif average >= 60:
    print("Grade: First Class")
elif average >= 40:
    print("Grade: Pass")
else:
    print("Grade: Fail")
```

8. Test Cases

Input / Scenario	Expected Result	Evidence in supplied run
20.5, 26.5, 29.7	Average 25.57; improved; fail	Same result (shown)
80, 70, 90	Average 80; improved; distinction	Not shown
60, 60, 40	Average 53.33; declined; pass	Not shown

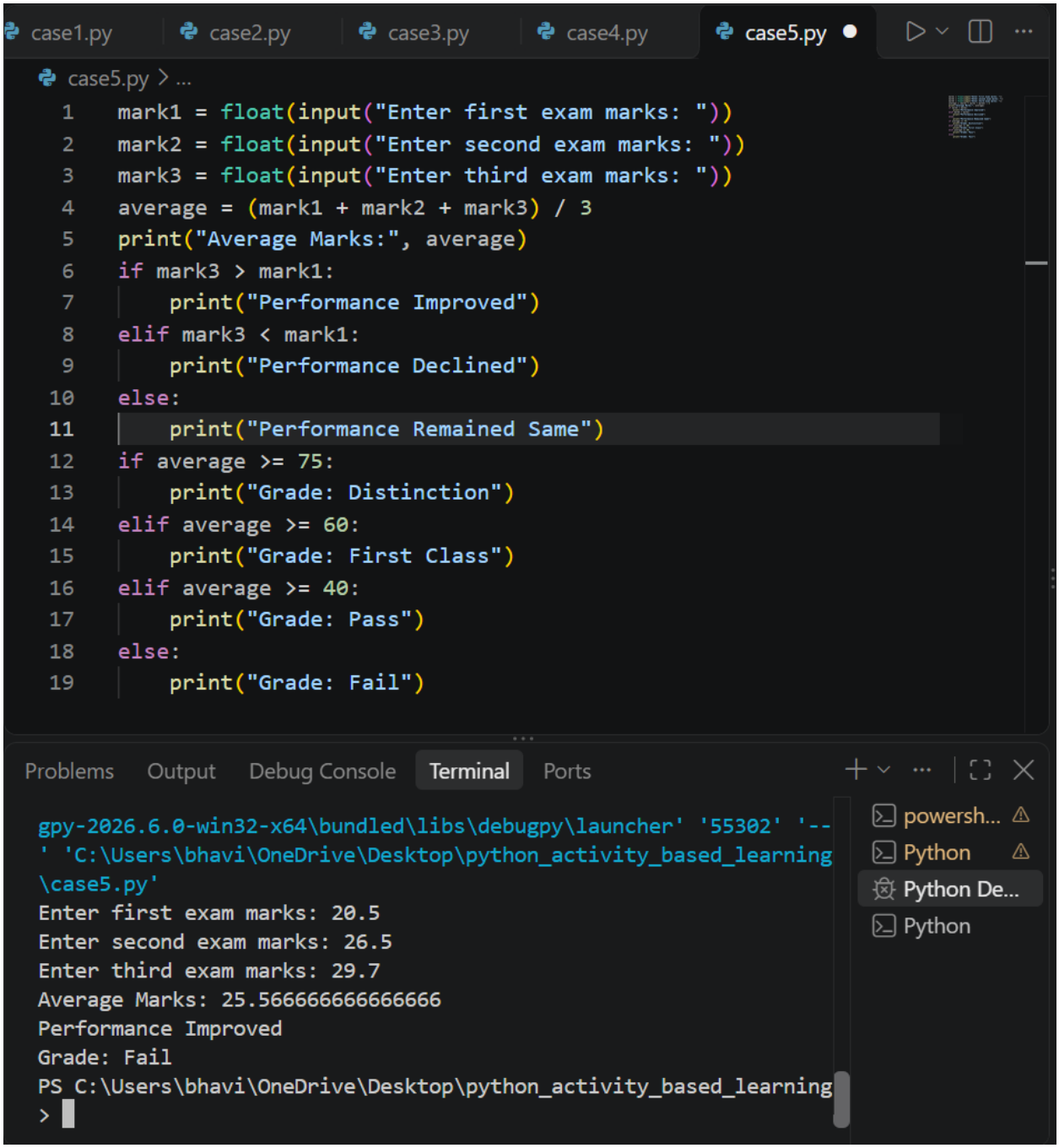
10. Conclusion

The program uses arithmetic calculations and conditional statements to summarize academic performance and assign a grade.

CASE STUDY 05: EXECUTION OUTPUT

9. Output Screenshot

The following image is the execution evidence supplied for this case study.



CASE STUDY 06: Shopping Budget Checker

1. Problem Statement

Accept a shopping budget and the prices of three products. Calculate the total shopping cost and determine whether the purchase is within budget. If the cost exceeds the budget, calculate the excess amount.

2. Objectives

• Read the budget and three product prices.
• Calculate the total cost.
• Compare the total with the budget.
• Display the excess amount when applicable.

3. Assumptions

• All monetary values are non-negative decimal numbers.
• Exactly three products are considered.
• No taxes, delivery fees, or discounts are included.

4. Algorithm

1. Read the shopping budget.
2. Read the prices of three products.
3. Add the prices to obtain the total.
4. If total ≤ budget, display within budget.
5. Otherwise calculate extra = total – budget and display the excess.

5. Flowchart

```
 START → Read budget and 3 prices → total = price1 + price2 + price3
 → total ≤ budget? → Yes: Within budget
                     No: extra = total − budget; Display excess
 → END
```

6. Pseudocode

```
BEGIN
  INPUT budget, price1, price2, price3
  total = price1 + price2 + price3
  PRINT total
  IF total <= budget THEN
    PRINT shopping is within budget
  ELSE
    extra = total - budget
    PRINT shopping exceeds budget and extra
  END IF
END
```

7. Python Source Code

```
budget = float(input("Enter your budget: "))
price1 = float(input("Enter first product price: "))
price2 = float(input("Enter second product price: "))
price3 = float(input("Enter third product price: "))

total = price1 + price2 + price3
print("Total Shopping Cost:", total)

if total <= budget:
    print("Shopping is within budget")
else:
    print("Shopping exceeds budget")
    extra = total - budget
    print("Amount exceeding budget:", extra)
```

8. Test Cases

Input / Scenario	Expected Result	Evidence in supplied run
Budget 500.25; prices 50.67, 70.65, 100.87	Total 222.19; within budget	Same result (shown)
Budget 200; prices 100, 80, 50	Total 230; exceeds by 30	Not shown
Budget 300; prices 100, 100, 100	Total 300; within budget	Not shown

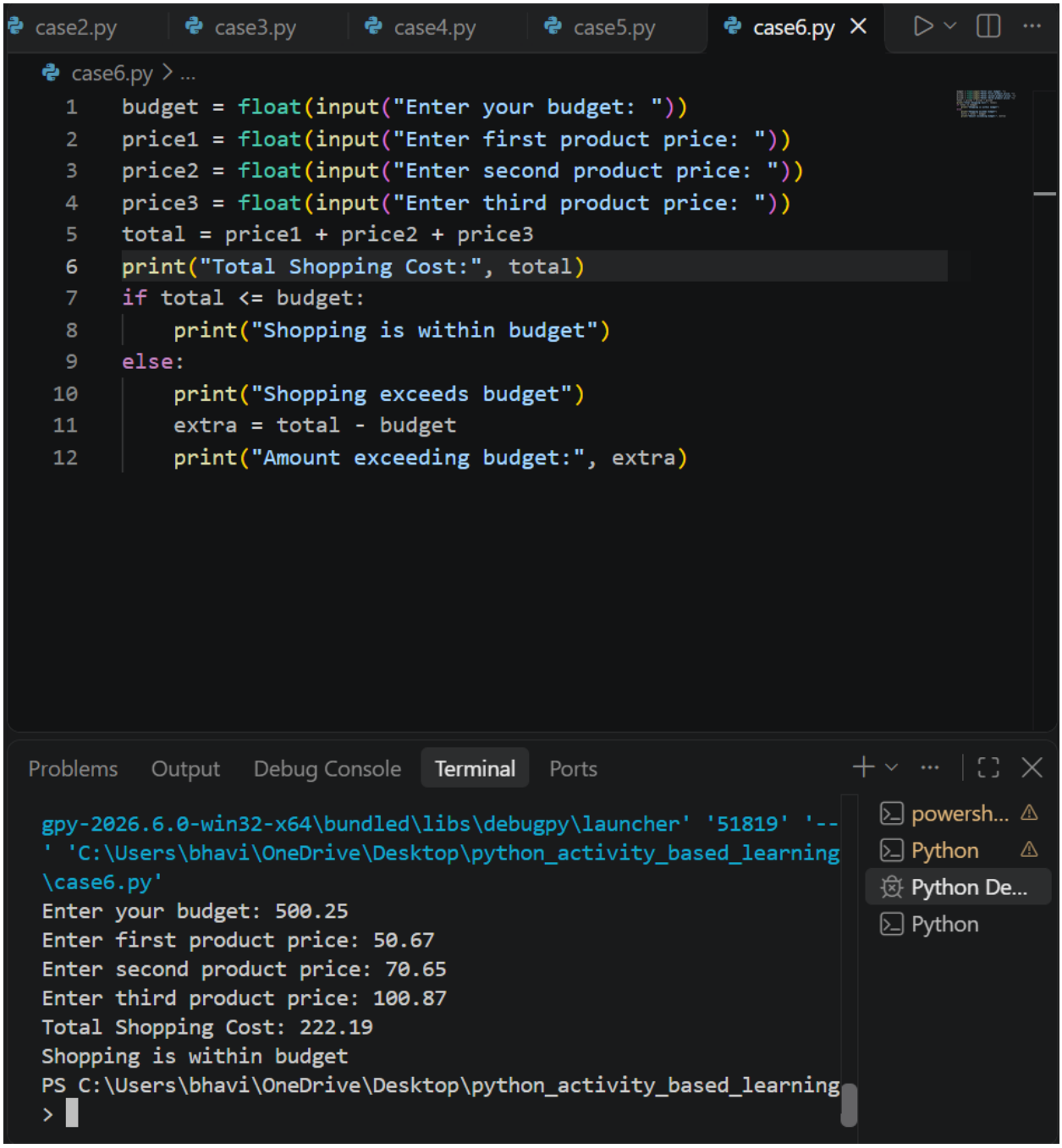
10. Conclusion

The program applies arithmetic and comparison operations to help determine whether a set of purchases fits a given budget.

CASE STUDY 06: EXECUTION OUTPUT

9. Output Screenshot

The following image is the execution evidence supplied for this case study.



CASE STUDY 07: Number Pattern Using Nested Loops

1. Problem Statement

Print an ascending number triangle followed by a descending number triangle for a user-specified number of rows. Reject zero or negative row counts.

2. Objectives

• Read the number of rows.
• Validate that the row count is positive.
• Use nested for loops to print an ascending triangle.
• Use nested for loops to print a descending triangle.

3. Assumptions

• The row count is a positive integer.
• The pattern starts at 1 and uses consecutive integers in each row.

4. Algorithm

1. Read rows.
2. If rows ≤ 0, display a positive-number message.
3. For i from 1 through rows, print numbers from 1 through i, then move to a new line.
4. For i from rows−1 down to 1, print numbers from 1 through i, then move to a new line.

5. Flowchart

```
START → Read rows → rows ≤ 0? → Yes: Print positive-number message → END
                                  No: i = 1 to rows → print 1..i
                                      i = rows−1 down to 1 → print 1..i
                                      → END
```

6. Pseudocode

```
BEGIN
  INPUT rows
  IF rows <= 0 THEN
    PRINT enter a positive number
  ELSE
    FOR i = 1 TO rows
      FOR j = 1 TO i: PRINT j
      PRINT newline
    FOR i = rows - 1 DOWN TO 1
      FOR j = 1 TO i: PRINT j
      PRINT newline
  END IF
END
```

7. Python Source Code

```
rows = int(input("Enter number of rows: "))

if rows <= 0:
    print("Enter a positive number")
else:
    for i in range(1, rows + 1):
        for j in range(1, i + 1):
            print(j, end=" ")
        print()

    for i in range(rows - 1, 0, -1):
        for j in range(1, i + 1):
            print(j, end=" ")
        print()
```

8. Test Cases

Input / Scenario	Expected Result	Evidence in supplied run
5	Ascending 1 to 5, then descending 4 to 1	Pattern shown
1	Single row: 1	Not shown
0	Enter a positive number	Not shown

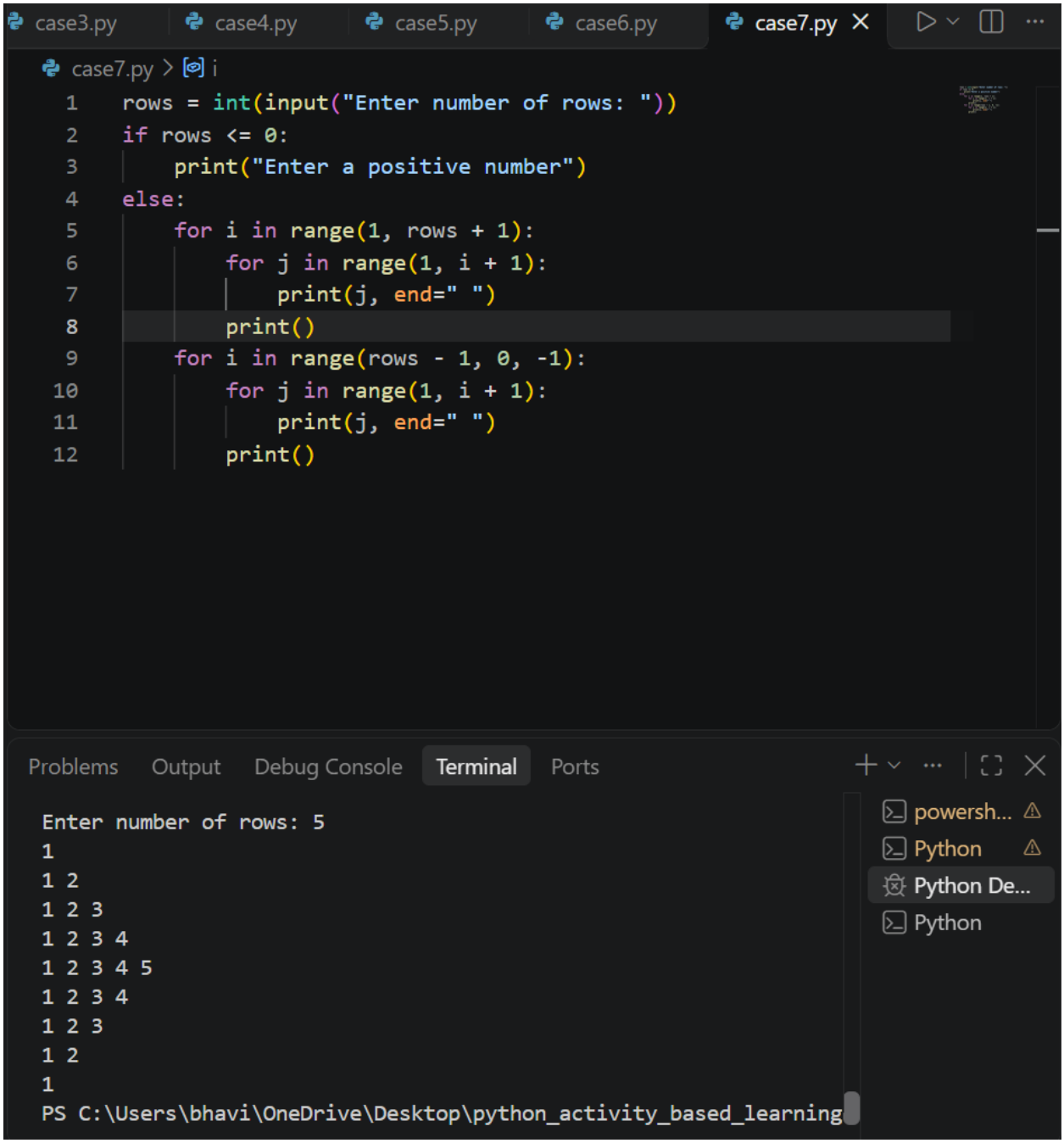
10. Conclusion

The program demonstrates nested for loops, range boundaries, and formatted output to generate a symmetric number pattern.

CASE STUDY 07: EXECUTION OUTPUT

9. Output Screenshot

The following image is the execution evidence supplied for this case study.



CASE STUDY 08: Password Validation

1. Problem Statement

Validate a password against four basic rules: at least eight characters, at least one uppercase letter, at least one lowercase letter, and at least one digit. Display the first unmet requirement or confirm validity.

2. Objectives

• Accept a password string.
• Check its length.
• Check for uppercase, lowercase, and numeric characters.
• Display a clear validation message.

3. Assumptions

• The password is treated as a string.
• The program checks the four listed rules only; it does not check special characters or common-password strength.
• The first failed rule is reported.

4. Algorithm

1. Read the password.
2. If its length is below eight, display the minimum-length message.
3. Otherwise check for an uppercase letter.
4. Then check for a lowercase letter.
5. Then check for a digit.
6. If all checks pass, display that the password is valid.

5. Flowchart

```
START → Read password → length < 8? → uppercase missing?
→ lowercase missing? → digit missing? → Print first failed rule
→ If all checks pass: Password is Valid → END
```

6. Pseudocode

```
BEGIN
  INPUT password
  IF length(password) < 8 THEN PRINT length requirement
  ELSE IF no uppercase character THEN PRINT uppercase requirement
  ELSE IF no lowercase character THEN PRINT lowercase requirement
  ELSE IF no digit THEN PRINT number requirement
  ELSE PRINT password is valid
END
```

7. Python Source Code

```
password = input("Enter your password: ")

if len(password) < 8:
    print("Password must contain at least 8 characters")
elif not any(ch.isupper() for ch in password):
    print("Password must contain an uppercase letter")
elif not any(ch.islower() for ch in password):
    print("Password must contain a lowercase letter")
elif not any(ch.isdigit() for ch in password):
    print("Password must contain a number")
else:
    print("Password is Valid")
```

8. Test Cases

Input / Scenario	Expected Result	Evidence in supplied run
Bhavitha@1234	All four checks pass	Password is Valid (shown)

Input / Scenario	Expected Result	Evidence in supplied run
abc123	Length requirement	Not shown
abcdefgh1	Uppercase requirement	Not shown
ABCDEF12	Lowercase requirement	Not shown

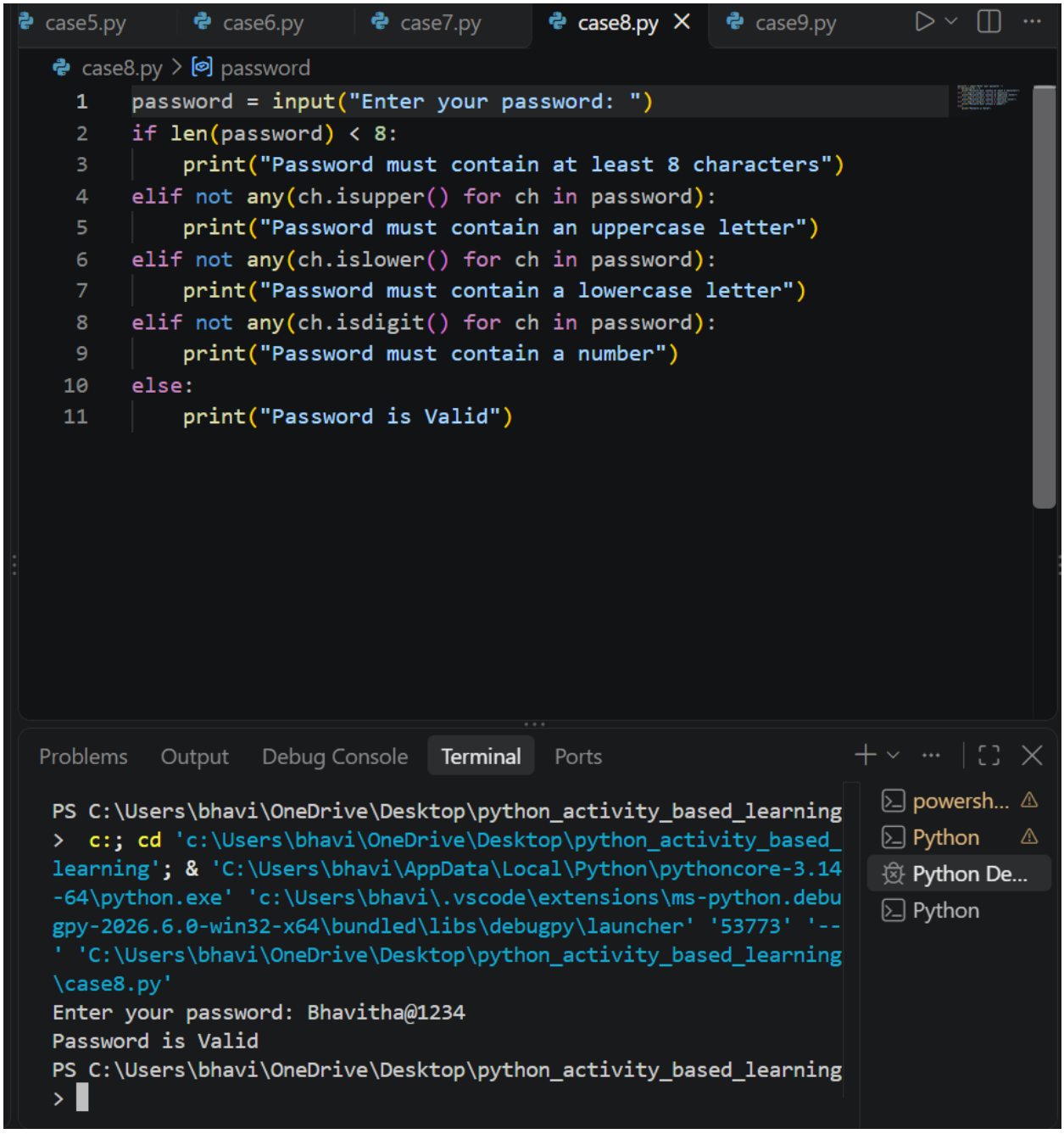
10. Conclusion

The program uses string methods and the any() function with conditional statements to validate basic password requirements.

CASE STUDY 08: EXECUTION OUTPUT

9. Output Screenshot

The following image is the execution evidence supplied for this case study.



CASE STUDY 09: Student Marks and Top Performer

1. Problem Statement

Collect marks in Python, Mathematics, and English for three students. Calculate each student's total and average marks, then identify the student with the highest total.

2. Objectives

• Store student names in a list.
• Collect three subject marks for each student.
• Use a nested list to store marks.
• Calculate totals and averages.
• Identify the top performer.

3. Assumptions

• Exactly three students and three subjects are considered.
• Marks are entered as integers and are expected to be within the subject's valid range.
• The top performer is determined by the highest total; ties are resolved in favor of the first student in the list.

4. Algorithm

1. Create a list of three student names and an empty marks list.
2. For each student, read three subject marks and append them as a row.
3. For each row, calculate the total and average and display them with the student's name.
4. Create a list of row totals.
5. Find the index of the maximum total and display the corresponding student.

5. Flowchart

```
START → Create student list and empty marks list → Read 3 subjects per student
→ Store each student's marks → Loop through marks: calculate total and average
→ Build totals list → Find index of maximum total → Print top performer → END
```

6. Pseudocode

```
BEGIN
  students = [Student 1, Student 2, Student 3]
  marks = empty list
  FOR each student IN students
    INPUT mark1, mark2, mark3
    APPEND [mark1, mark2, mark3] TO marks
  FOR i from 0 to length(students)-1
    total = sum(marks[i])
    average = total / 3
    PRINT student, total, average
  totals = sum of each row in marks
  top = index of maximum value in totals
  PRINT students[top]
END
```

7. Python Source Code

```
students = ["Student 1", "Student 2", "Student 3"]
marks = []

for student in students:
    mark1 = int(input("Enter Python marks: "))
    mark2 = int(input("Enter Maths marks: "))
    mark3 = int(input("Enter English marks: "))
    marks.append([mark1, mark2, mark3])

for i in range(len(students)):
    total = sum(marks[i])
    average = total / 3
    print(students[i])
    print("Total Marks:", total)
    print("Average Marks:", average)

totals = [sum(row) for row in marks]
top = totals.index(max(totals))
print("Top Performer:", students[top])
```

8. Test Cases

Input / Scenario	Expected Result	Evidence in supplied run
Student totals 120, 120, 140	Student 3 is top performer	Student 3 (shown)
Totals 150, 180, 160	Student 2 is top performer	Not shown
All totals equal	Student 1 (first maximum)	Not shown

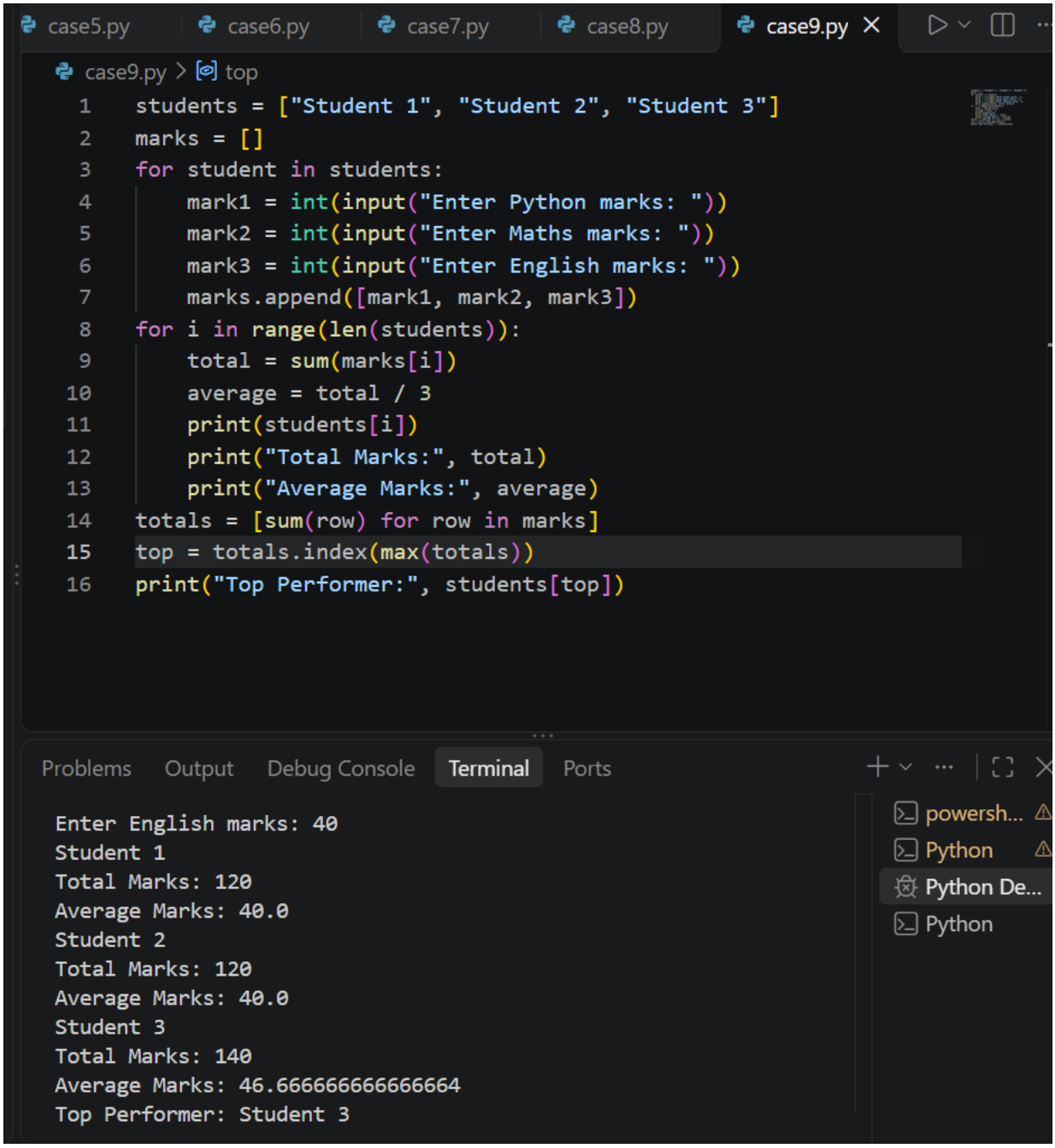
10. Conclusion

The program demonstrates lists, nested lists, loops, sum(), max(), and index() to process student marks and identify the highest scorer.

CASE STUDY 09: EXECUTION OUTPUT

9. Output Screenshot

The following image is the execution evidence supplied for this case study.



CASE STUDY 10: Canteen Ordering and Stock Management

1. Problem Statement

Create a simple canteen ordering program with two menu items: Idli at Rs. 30 and Dosa at Rs. 50. Accept a menu choice and quantity, check stock availability, calculate the bill, and update the selected item's stock.

2. Objectives

• Initialize stock for Idli and Dosa.
• Display the menu and accept a choice and quantity.
• Validate quantity and menu choice.
• Check stock before confirming an order.
• Calculate the bill and update stock.

3. Assumptions

• Initial stock is 20 Idlis and 15 Dosas.
• Idli costs Rs. 30 and Dosa costs Rs. 50.
• Only one item type is ordered per run.
• Quantity must be positive and cannot exceed available stock.

4. Algorithm

1. Initialize Idli stock to 20 and Dosa stock to 15.
2. Display the two menu options.
3. Read the choice and quantity.
4. If quantity is non-positive, display invalid quantity.
5. If choice is 1, check Idli stock; if sufficient, calculate quantity × 30, reduce stock, and display bill and remaining stock.
6. If choice is 2, check Dosa stock; if sufficient, calculate quantity × 50, reduce stock, and display bill and remaining stock.
7. If stock is insufficient or choice is invalid, display the appropriate message.

5. Flowchart

```
START → Initialize stock → Display menu → Read choice and quantity
→ quantity ≤ 0? Invalid quantity
→ choice = 1? Check Idli stock → Bill = quantity × 30; update stock
→ Else choice = 2? Check Dosa stock → Bill = quantity × 50; update stock
→ Else: Invalid choice → END
```

6. Pseudocode

```
BEGIN
  idli_stock = 20; dosa_stock = 15
  DISPLAY menu with prices
  INPUT choice, quantity
  IF quantity <= 0 THEN PRINT invalid quantity
  ELSE IF choice = 1 THEN
    IF quantity <= idli_stock THEN
      bill = quantity * 30
      idli_stock = idli_stock - quantity
      PRINT bill, remaining idli stock
    ELSE PRINT insufficient idli stock
  ELSE IF choice = 2 THEN
    IF quantity <= dosa_stock THEN
      bill = quantity * 50
      dosa_stock = dosa_stock - quantity
      PRINT bill, remaining dosa stock
    ELSE PRINT insufficient dosa stock
  ELSE PRINT invalid choice
END
```

7. Python Source Code

```
idli_stock = 20
dosa_stock = 15

print("1. Idli - Rs. 30")
print("2. Dosa - Rs. 50")
choice = int(input("Enter your choice: "))
quantity = int(input("Enter quantity: "))

if quantity <= 0:
    print("Invalid quantity")
elif choice == 1:
    if quantity <= idli_stock:
        bill = quantity * 30
        idli_stock = idli_stock - quantity
        print("Your Bill:", bill)
        print("Remaining Idli Stock:", idli_stock)
    else:
        print("Insufficient Idli Stock")
elif choice == 2:
    if quantity <= dosa_stock:
        bill = quantity * 50
        dosa_stock = dosa_stock - quantity
        print("Your Bill:", bill)
        print("Remaining Dosa Stock:", dosa_stock)
    else:
        print("Insufficient Dosa Stock")
else:
    print("Invalid Choice")
```

8. Test Cases

Input / Scenario	Expected Result	Evidence in supplied run
Choice 50; quantity 7	Invalid choice	Invalid Choice (shown)
Choice 1; quantity 2	Bill Rs. 60; Idli stock 18	Not shown
Choice 2; quantity 16	Insufficient Dosa Stock	Not shown
Choice 2; quantity 3	Bill Rs. 150; Dosa stock 12	Not shown

10. Conclusion

The program demonstrates menu-driven selection, nested conditions, arithmetic billing, and stock updates. The supplied execution screenshot tests the invalid-choice branch.

CASE STUDY 10: EXECUTION OUTPUT

9. Output Screenshot

The following image is the execution evidence supplied for this case study.

